Chapter-3
Large language Models

Model Selection

It means choosing the most suitable LLM for a particular application based on factors such as performance, knowledge cutoff date, on-premise versus cloud based hosting, open source, open weight and propietary model, price, context window, latency.

1. Performance:

It refers to how accurately and effectively a model performs a given task, such as text generation, summarization, reasoning, coding, or question answering.

2. **Knowledge cutoff date:** is the date up to which the training data of a model was finalized.

The knowledge cutoff date indicates the approximate latest date of information included in a model's training data.

- A model with an earlier cutoff may not know about recent events or developments.
- For current information, a model may require web search, retrieval-augmented generation (RAG), or access to updated databases.
- A cutoff date does not necessarily indicate when the model was last updated.

3. On-premise versus cloud based hosting:

Cloud-based: The model is hosted by an external provider and accessed through a web application or API.

On-premise: The model runs on the organization's own hardware/network.

4. Open source, open weight and propietary model:

1. Proprietary Model: A model controlled by a company and generally accessed through a **web application or API (**Application Programming Interface**)**.

2. Open-Weight Model: The trained **model weights are made available**, allowing users to download and run the model on their own hardware, subject to its license. However, other information such as training data may not be fully disclosed.

3. Open-Source Model

The source emphasizes that truly open-source models provide broader transparency, including details such as:

- Model architecture
- Training data
- Other underlying details

5. Price:

The **cost of using an LLM** is another important model-selection factor.

Before selecting a model, estimate:

Number of API requests × Tokens processed per request × Price per token → **Estimated total cost**

6. Context window:

The context window is the maximum amount of information a model can process or consider in a single interaction, measured in tokens.

- It may include input text, conversation history, instructions, and generated output, depending on the model.
- A larger context window helps with long documents, lengthy conversations, and multiple files.

7. Latency:

Latency is the time taken by a model to begin responding or to complete a response.

- Lower latency is important for real-time applications.
- It depends on model size, hardware, network conditions, server load, and output length.
- Time to first token and total response time are two useful latency measures.

Lower latency = Faster response

Prompt Templates

A **prompt** is the instruction, question, or text given to an AI model (such as ChatGPT) to tell it what to do or what response to generate.

Prompt template: A reusable, structured format for preparing instructions and user inputs before sending them to a Large Language Model (LLM).

ChatPromptTemplate

ChatPromptTemplate allows us to create a prompt using a list of messages, such as:

- System message: Defines the role, behavior, or instructions for the model.
- Human/User message: Contains the actual user query or task.

Example: **Prompt template: Translate "{sentence}" into {language}.**

If we supply sentence = "I love programming" and language = "German", the resulting prompt becomes:

Translate "I love programming" into German.

The values inside the curly braces are replaced with actual inputs before the prompt is sent to the model.

```
from langchain_core.prompts import ChatPromptTemplate
prompt_template = ChatPromptTemplate.from_messages([
   ("system", "You are a language translation assistant."),
   ("human", "Translate '{input}' into {target_language}.")
])
result = prompt_template.invoke({
   "input": "I love programming.",
   "target_language": "German"
})
print(result)
```

Expected prompt content:

- System: You are a language translation assistant.
- Human: Translate 'I love programming.' into German.

Term	Meaning
Placeholder	A variable written inside curly braces, such as {input}
from_messages()	Creates a chat prompt template from messages
invoke()	Supplies actual values for the placeholders
Dictionary	Maps placeholder names to their corresponding values

2. Improving Prompts Using LangChain Hub

What is LangChain Hub?

LangChain Hub is a platform for discovering and reusing prompt templates created by other users. The supplied material illustrates how an existing prompt can be used to generate a more detailed prompt. i.e., A short or incomplete instruction is expanded into a detailed prompt with a role, task instructions, context, and desired output.

Example:

- Input prompt: “summer, vacation, beach”
- Task: “Shakespeare poem”
- Improved prompt: Generate a 14-line sonnet about summer, vacation, and the beach, following a specified rhyme scheme and poetic style.

The improved prompt gives the model more specific guidance and constraints.

Chains in LangChain

Chain is a **concept.**
LangChain is a **tool/framework** to build that concept.

1. Chain?

- A chain is a sequence where output of one step becomes input of next step.
- So Chain is just a **workflow pattern**. It can be of 4 types we discussed: Sequential, Parallel, Router, With Memory.

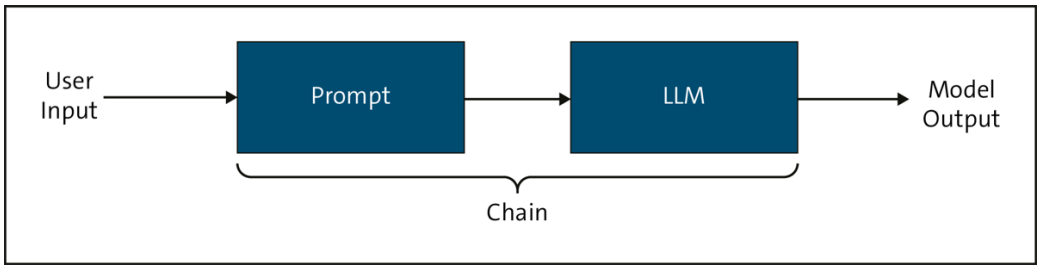
2. LangChain

- LangChain is an open-source framework (a software library in Python/JS) that helps developers easily build these chains.
- Without LangChain, you have to write a lot of code to connect LLM + memory + database + tools.

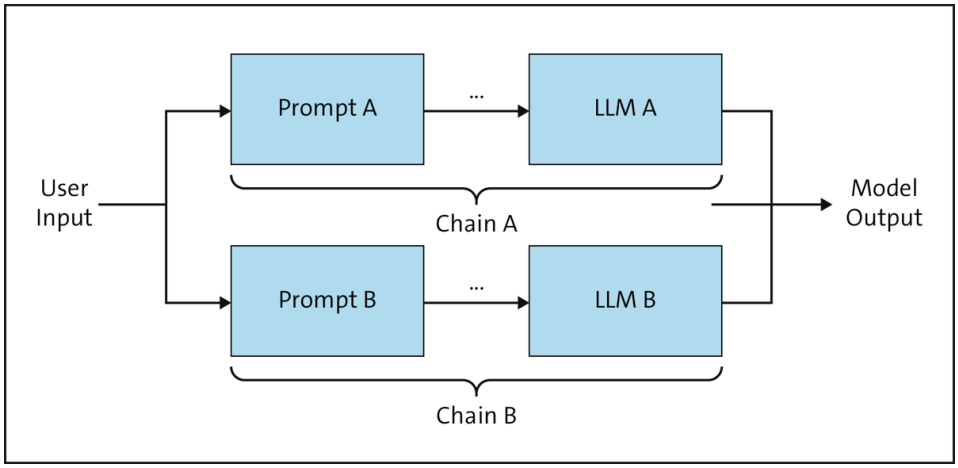
A chain is a sequence of connected processing components that work together to accomplish a task.

The output of one component becomes the input of the next component.

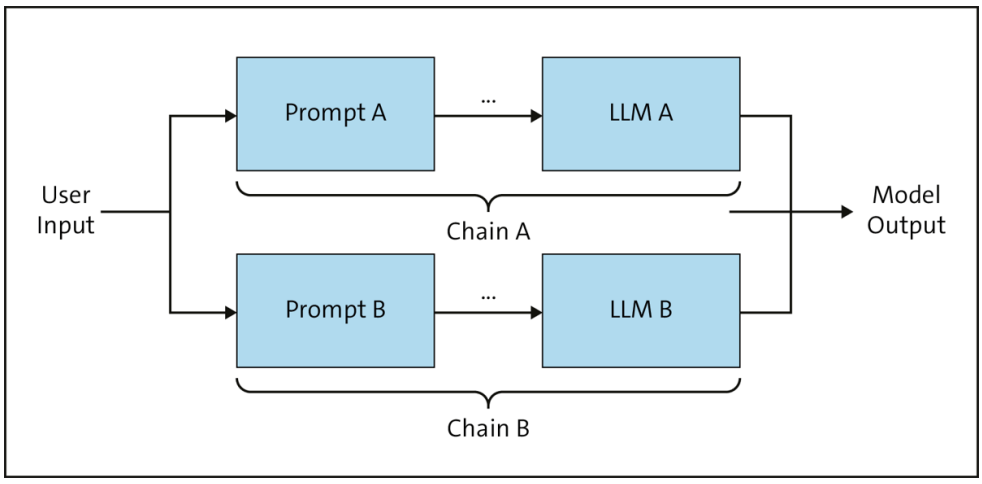
- A chain may contain prompts, LLMs, output parsers, retrievers, and tools.
- The material introduces four important types of chain implementations.
  1. **Sequential Chain (Linear Process):** A sequential chain processes components one after another.
     - Like an assembly line: A → B → C → D
     - One step finishes, next starts.
     - Example: Washing clothes: Collect → Wash → Dry → Iron



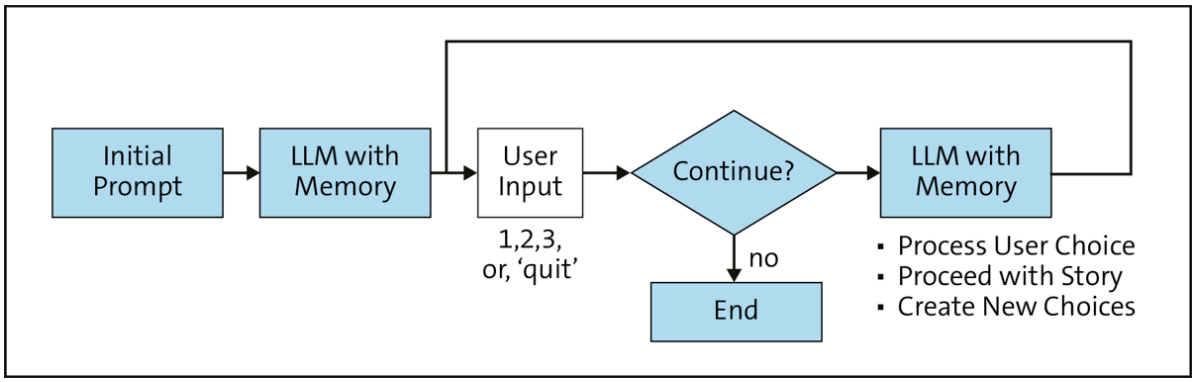
  2. **Parallel Chain (Concurrent Work):** A parallel chain executes two or more independent processing chains simultaneously and combines their results.
     - One task splits into many tasks that run at same time, then merge.
     - Example: Group project - 3 students research 3 topics together, then combine report.



3. **Router Chain(Decision / Switchboard):** A router chain selects an appropriate processing path according to the input.
    - Input goes to a Router which decides which path to take.
    - Example: Railway station inquiry - If you ask for Platform? → Path A, Ticket? → Path B, Train time? → Path C



4. **Chain with Memory(Learns from Past):** A chain with memory maintains information about previous interactions so that subsequent responses can use the conversation history.
    - Chain stores past output in memory and uses it for next decision. Has a feedback loop.

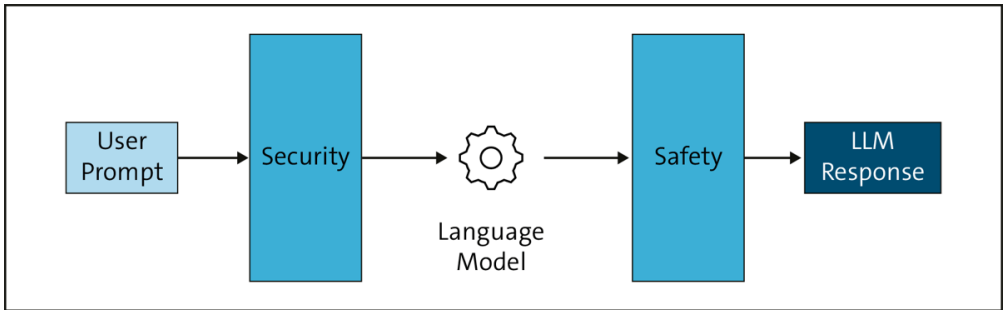


3.8 Safety and Security

When deploying an LLM system, we need to protect **both the system and the users**.

Security → Protect the LLM from harmful/malicious inputs
Safety → Protect users from harmful outputs

Simple Pipeline of LLM Safety and Security



Some important aspects of security or important security threats include the following:

3.8.1 Security

LLM security protects the model from malicious prompts, attacks, and vulnerabilities that could compromise its integrity or confidentiality.

1. Hackers / Malicious Actors

Hackers may try to trick or attack the LLM to make it behave incorrectly or reveal sensitive information.

> **Example:** A user gives a specially designed prompt to make the LLM reveal information that it should not disclose.

2. Data Poisoning

Data poisoning means intentionally adding incorrect or harmful data to the training data so that the model learns unwanted behavior.

> **Example:** If incorrect or biased information is added to training data, the model may learn and reproduce that bias.
>
> **Easy memory:** Bad training data → Bad learned behavior

3. Robustness

Robustness means the LLM should continue to work correctly even when someone tries to confuse, manipulate, or attack it.

> **Example:** The model should not easily change its intended behavior just because a user gives a tricky or malicious prompt.

3.8.2 Safety

1. Bias

Bias means the LLM may produce unfair or one-sided answers because of biases present in its training data.

Example: A model repeatedly associates a particular job only with one gender.
Goal: Reduce harmful or unfair responses.

2. Hallucinations

Hallucination occurs when an LLM gives incorrect or made-up information as if it were true.

Example: The model gives a fake research paper or an incorrect fact with confidence.
Goal: Reduce incorrect or unsupported information.

3. Personally Identifiable Information (PII)

PII is personal information that can identify a particular person. The LLM should not unnecessarily reveal or leak such information.

Examples: Name, phone number, address, identification details.
Goal: Protect people's personal information.

4. Ethics

Ethics means ensuring that the LLM behaves responsibly and does not provide harmful recommendations or misinformation.

Example: The model should avoid giving advice that could cause harm.
Goal: Make AI responsible and trustworthy.

5. Off-topic

An LLM should normally answer questions related to the purpose of the application and avoid unrelated topics.

Example: A **college admission chatbot** should answer questions about admission, courses, and fees rather than unrelated questions.
Goal: Keep the chatbot focused on its intended purpose.

6. Reputational Damage

Unsafe or inappropriate responses from an LLM can damage the reputation of the organization using it.

Example: If a company's chatbot gives offensive or inappropriate answers, users may lose trust in the company.
Goal: Protect user trust and the organization's reputation.

7. Robustness

The LLM should resist attempts to manipulate it into revealing secrets or generating harmful content.

Example: A user repeatedly tries different prompts to make the model reveal confidential information. A robust system should resist these attempts.
Goal: Prevent manipulation and misuse.

8. Context

The LLM should give answers that are appropriate for the specific purpose and context of the application.

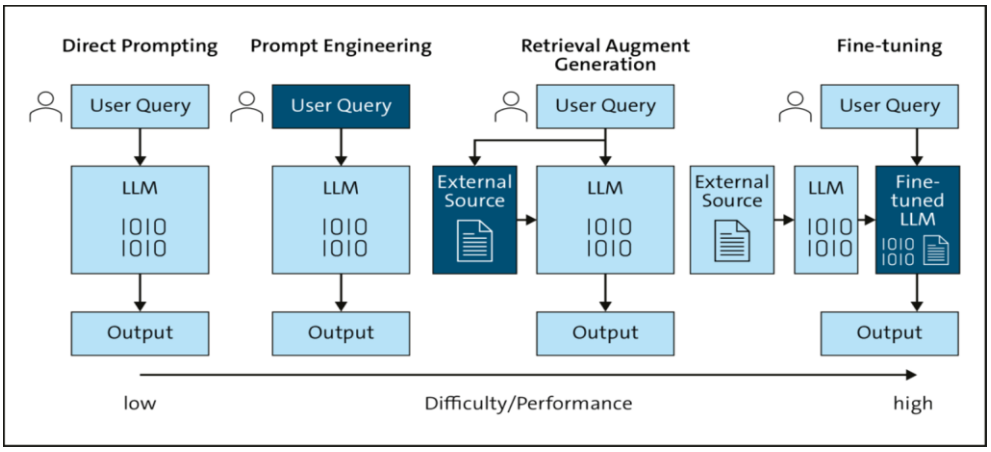
Example: **Medical chatbot** → Medical-related information
Financial chatbot → Financial-related information
A medical chatbot should not unnecessarily provide investment advice.

3.9 Model Improvements

Sometimes an LLM does not give the desired result even when the prompt is reasonable. In such cases, we can **improve the way the LLM is used** rather than immediately changing the model.

Model Improvement = Applying techniques to improve the quality of LLM outputs.

The approaches can be arranged from **simple to more difficult**.



1. Direct Prompting

✓ Direct prompting means simply giving a query or instruction to the LLM and using its response. It is simple and easy, often produces good results. But may not always provide the required output.

Example: "Summarize this article."

2. Prompt Engineering

✓ Prompt engineering means improving the prompt by providing clearer and more detailed instructions so that the LLM better understands what we want.

Instead of: "Summarize this article."
Use: "Summarize this article in 5 bullet points using simple language and include only the main findings."

3. RAG — Retrieval-Augmented Generation

✓ Sometimes prompt engineering is not enough because the required information is not available in the LLM's existing knowledge**.**

✓ RAG enriches the LLM's knowledge at runtime by providing relevant documents along with the user's query.

4. Retraining / Updating Model Weights

✓ Another approach is to integrate the missing information into the model's weights**.**
✓ This requires at least partial retraining of the LLM.

Simple idea

New/Missing Data → Retraining / Fine-tuning → Updated Model Weights → Improved LLM

Prompt Engineering → Change the instruction
RAG → Add external information at runtime
Retraining → Change/update the model's learned weights

LLM gives unsatisfactory answer
↓
Improve the Prompt (***If still not enough***)
↓
Provide Relevant Documents using RAG (***If required in specific cases***)
↓
Retrain/Update Model Weights

3.10 New Trends in LLMs

The content discusses three important trends in Large Language Models (LLMs):

1. Reasoning Models

Reasoning models are LLMs designed to work through intermediate steps before producing a final answer, particularly for complex tasks.

- Traditional LLMs may generate an answer directly, sometimes struggling with complex reasoning.
- Reasoning models can work through intermediate steps and explore possible solutions.

2. Small Language Models (SLMs)

Small Language Models are language models with fewer parameters than large LLMs, generally requiring less memory and computational power.

- SLMs can be trained for specific use cases using carefully selected datasets.
- They need fewer computational resources for training and operation.
- They may be suitable for local devices, including laptops and smartphones.

3. Test-Time Compute

It refers to the computational resources used during inference to help a trained model solve a problem. (i.e., instead of spending all the effort during training, the model uses additional computation while answering a question.)

Chapter-4
Prompt Engineering

4.1.1 Prompt Process

A **prompt** acts as a bridge between human intention and AI-generated responses.

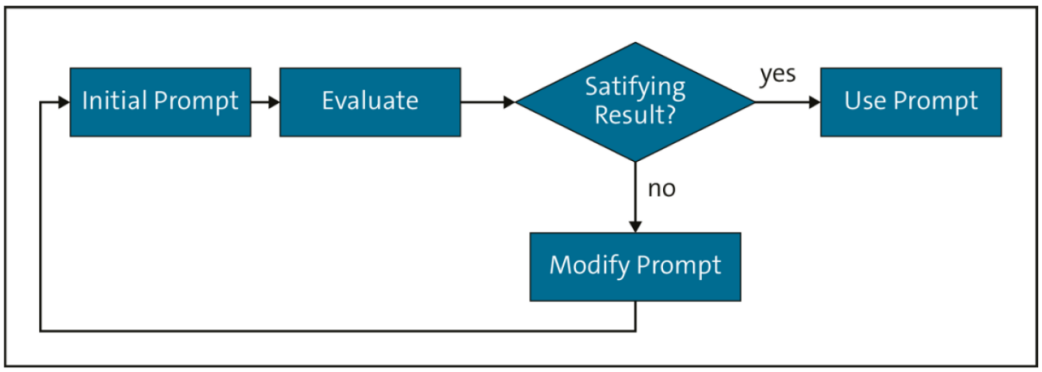
Prompt engineering is the process of designing and refining instructions given to an AI model to obtain the desired output.



Prompt	Prompt Template
A specific instruction given to an AI model.	A reusable structure containing variables.
Contains actual information.	Variables are replaced with actual values when invoked.
Example: “Explain machine learning.”	Example: “Explain `{TOPIC}` with an example.”

Prompt Engineering Process

Prompt engineering is an iterative process. We improve the prompt until the output meets our requirements.



4.1.2 Prompt Components

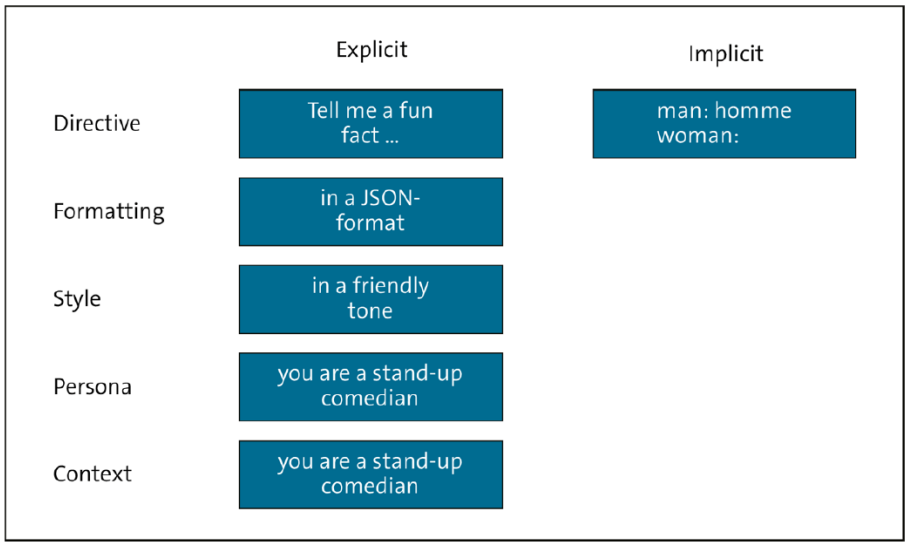
The components of a prompt can be explicit or implicit: explicit components directly state the instruction, output format, style, persona, or context, while implicit components allow the model to infer the intended task from examples or patterns.

For example, “Tell me a fun fact” is an explicit directive, whereas “man: homme, woman:” implicitly asks the model to complete the translation pattern.

A prompt can include the following components:

- Instruction: What should the model do?
- Context: What background information does the model need?
- Examples: What type of response is expected?

- Role or persona: What role should the model adopt?
- Output format: How should the response be presented?
- Style and tone: How should the response sound?



4.1.3 Basic Principles of Effective Prompting

1. Clear instructions: Give specific instructions rather than vague requests.

- Poor prompt: “Write about dogs.”
- Improved prompt: “Write a 300-word guide for first-time dog owners covering suitable breeds, exercise needs, grooming and estimated ownership costs.”

2. Task decomposition: Break a complex task into smaller, manageable subtasks.

Example: Writing a research paper can be divided into:

1. Define the research question.
2. Review relevant literature.
3. Collect and analyze data.
4. Write the paper.
5. Edit and finalize the paper.

3. Using delimiters: Delimiters separate different parts of a prompt, such as instructions, context and questions. Examples include triple quotes, XML tags and special characters.

Example:

```
<context>
The Great Wall's famous sections were built
during the Ming Dynasty.
</context>

<question>
During which dynasty were the famous sections built?
</question>
```

4. Requesting explanations: Ask for a solution with clear steps or an explanation of the method.

Example:

"Solve $3x^2+6x-24=0$. Identify the coefficients, show the calculation steps, and verify the solutions."

5. Leveraging personas: Assign a role to guide the model's response, expertise level or communication style. For example:

- "Act as a mathematics teacher."
- "Respond as a financial analyst."
- "Explain this concept as a beginner-friendly programming instructor."

6. Providing examples: Provide one or more examples of the expected input-output pattern.

Example:
```
English: Book → Spanish: Libro
English: Water → Spanish: Agua
English: House → Spanish:
```

The model can infer that the expected output is "Casa."

7. Output control: Specify the required length, format, style and level of detail. For example:

- "Answer in exactly three bullet points."
- "Summarize in 100 words."
- "Return the results in a table."
- "Return the data in JSON format."

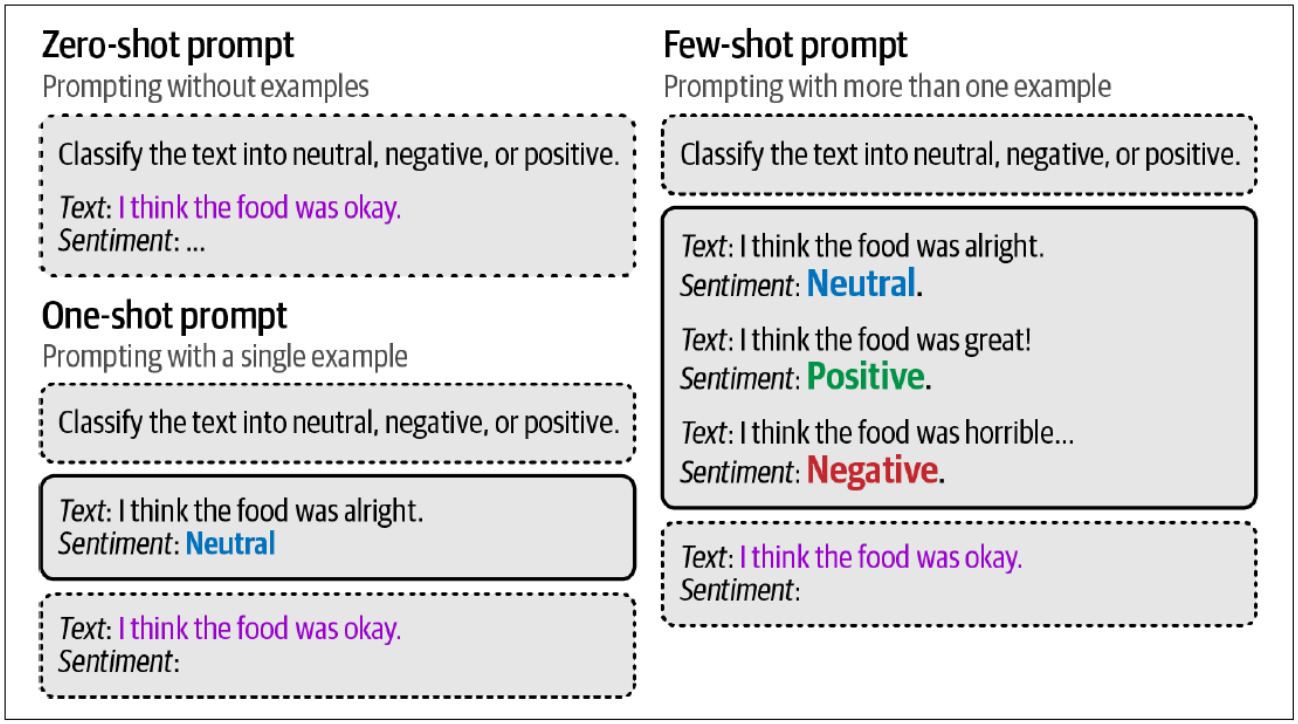
8. Multiple outputs and selection: Ask the model to generate multiple versions and select the most suitable one according to defined criteria.

Example: "Generate three titles for a research paper and select the clearest and most technically appropriate title."

Types of Prompting Techniques

1. Few-Shot Prompting

Providing a few examples in the prompt to help the AI understand the task and generate the expected answer.



2. Chain-of-Thought (CoT) Prompting (Think before answering)

Chain of Thought (CoT) prompting is a technique for improving the reasoning capabilities of LLMs by generating **intermediate reasoning steps**. This helps the LLM generate more accurate answers. i.e., break down a problem into sequential, logical steps *before* giving the final answer.

"Think step by step and explain how you got the answer."

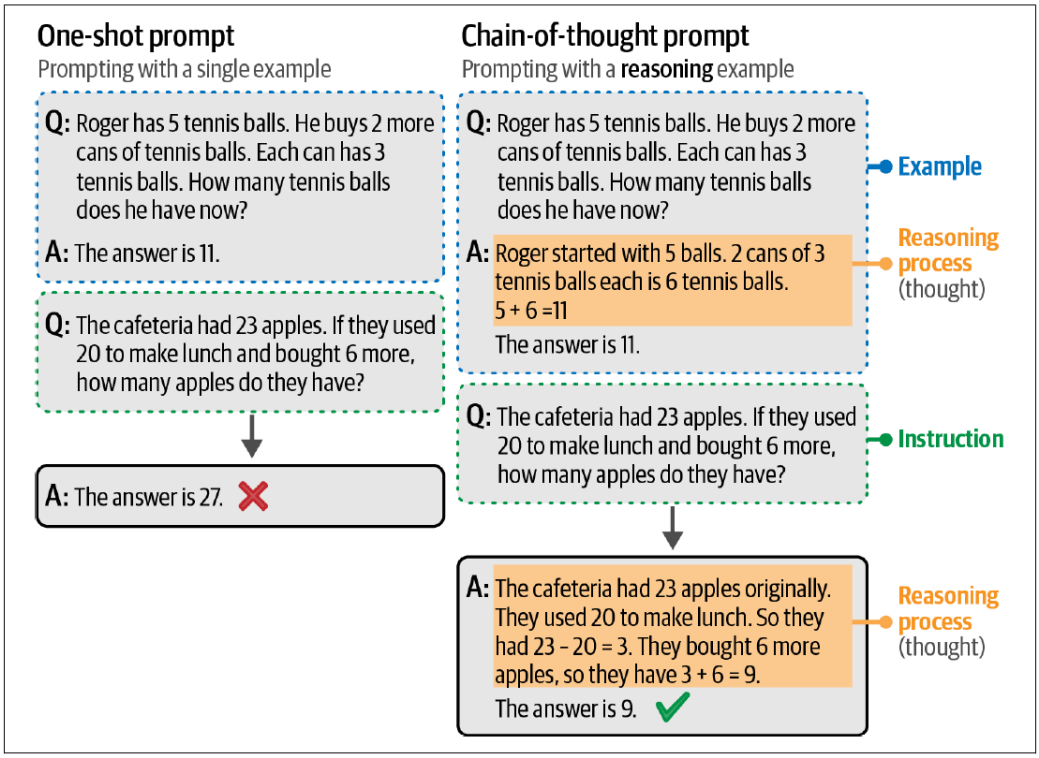
Improves tasks involving logic or calculations.

Prompt: When I was 3 years old, my partner was 3 times my age. Now, I am 20 years old. How old is my partner? Let's think step by step. (Ans. is **26 yrs old**)

Output: 1. Let's assume my current age is "x" years.

2. When I was 3 years old, my age was 3 years.
3. My partner's age at that time was 3 times my age, which means my partner's age was 3 * 3 = 9 years.
4. Now, I am 20 years old, which means my age has increased by 20 - 3 = 17 years since I was 3 years old.
5. Since my partner's age was 9 years when I was 3 years old, and my age has increased by 17 years, my partner's age must have also increased by 17 years.
6. Therefore, my partner's current age is 9 + 17 = 26 years.

So, my partner is 26 years old.

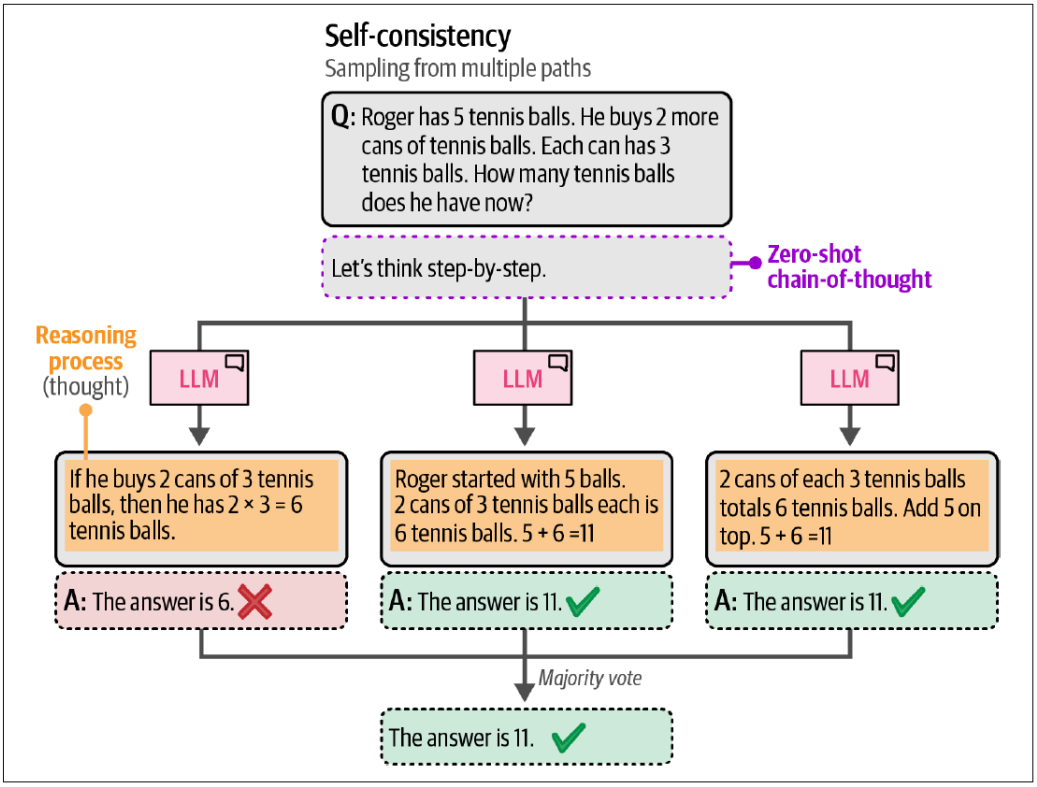


3. Self-Consistency Prompting (Sampling Outputs)

Self-consistency is a prompting technique in which the same prompt is given to a generative model multiple times to obtain different responses. The most frequently occurring answer is selected as the final answer to improve reliability and reduce the impact of randomness.

It provides an approximate indication of the likelihood that an answer is correct; however, generating multiple responses increases computational cost.

Generate multiple answers → choose the most consistent one.



4. Prompt Chaining

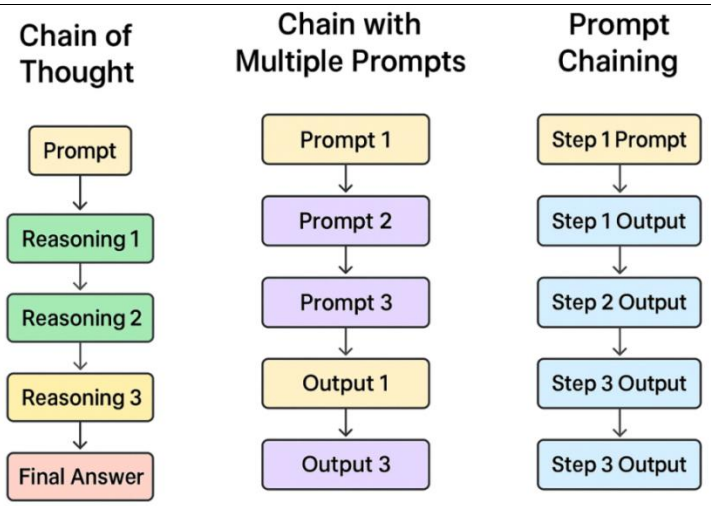
Prompt chaining connects multiple prompts in sequence, where the output of one step becomes context for the next step.

Break a big task into smaller steps and run sequential prompts.

Example:

1. Extract keywords
2. Generate summary
3. Create questions

	Chain of Thought	**Chain with Multiple Prompts (single template)**	**Prompt Chaining (Multiple template)**
Meaning	Chain of Thought is a method where the model produces intermediate reasoning steps (a chain of logical thoughts) to reach the final answer.	A **single task** is completed using **multiple prompts sent in one structured sequence**, often within **one model call** or **one prompt template**.	A full task is broken into **separate steps**, and the **output of one model call becomes the input of the next**.
Purpose	Ask model to think step-by-step	Multi-turn conversation stored in one template	Split complex task into stages
Number of Prompts	1	Multiple (Many blocks)	Multiple prompts
How It Handles the Task	Does all reasoning in **one prompt**	Multi-turn conversation; each prompt refines the task	Each step is an **independent prompt module**
# of model calls	1	1	Multiple
	Prompt (with "think step-by-step") ↓ (one call) LLM generates chain-of-thought + final answer	Prompt 1 + Prompt 2 (concatenated) ↓ (one combined call) LLM output	Prompt 1 → LLM → Output 1 Prompt 2 (using Output 1) → LLM → Output 2 Prompt 3 (using Output 2) → LLM → Final Output
Pros	Fast, simple	Easy to iterate, interactive	Highly controlled, good for automation
Cons	Harder to control steps, may hallucinate	Depends on conversation memory	Requires design effort; more steps
Example	"Explain your reasoning."	Phi-3 style prompt template	LangChain / DSPy pipeline

5. Self-feedback

Self-feedback is a technique in which an LLM evaluates its own generated output, provides feedback, and produces a revised version.

Workflow

- The model generates an initial response.
- It evaluates the response against specified criteria.
- It assigns a rating or score and provides feedback.
- It generates an improved version.
- The process repeats until a target score is reached or the maximum number of iterations is completed.